

# **AKSA-SDS Summer Internship**

**Name: Abdur Rehman Tariq**

**Dept: AI**

## **1. Introduction and Objective**

This report documents the design and development of an end to end e-commerce chatbot built for a Pakistani laptop retail website. The system scrapes structured product data from the website, stores it in a SQL database, prepares a frequently asked questions dataset from the website's published policies, and routes user queries to the correct source of information using a semantic router. The application is exposed through a Streamlit chat interface with a visible debug panel and made publicly accessible through ngrok. No agent framework or tool-calling system was used anywhere in the pipeline. Every routing and retrieval decision is made through plain, inspectable Python functions and cosine similarity comparisons, which keeps the system deterministic and easy to justify.

## **2. Website and Category Selection**

The website chosen for this project is priceoye.pk, a Pakistani electronics and gadgets retailer. The Laptops category was selected as the product domain because it offers a large, consistent catalogue with clearly structured attributes such as brand, processor, RAM, storage, and price, which map naturally onto SQL columns. Priceoye.pk was preferred over alternatives such as Daraz because its listing and product pages are server rendered, meaning the HTML returned by a plain request already contains the product information needed, without requiring a headless browser. This significantly simplified the scraping approach and made the pipeline more reliable for a graded submission with a live demonstration requirement.

The FAQ dataset was built from the same website rather than invented from scratch. Priceoye.pk publishes real policy pages covering returns, warranty, shipping, self pickup, and payment methods. These pages were used as the basis for the FAQ question and answer pairs, so the FAQ dataset reflects genuine store policy rather than fabricated content, which keeps the retrieval and evaluation grounded in real information.

## **3. Scraping Strategy**

The scraper is implemented using the requests library to fetch listing pages and BeautifulSoup to parse the returned HTML. It iterates through the Laptops category listing pages, extracts each product card, and pulls the product name, price, discount percentage, rating, review count, brand, and stock status directly from the listing text. Because the full technical specification table on individual product pages is not present in the static HTML returned by a plain request, processor, RAM, and storage details are instead extracted from the product title itself using regular expressions, since Pakistani e-commerce listings conventionally include these details in the title, for example a processor generation and a RAM and storage combination in parentheses.

A fixed delay is added between page requests and the crawl is capped at a small number of pages so the scraper behaves respectfully toward the website rather than sending rapid, unthrottled requests. Running the scraper against the live Laptops category produced 204 products across all listing pages, which were saved to a JSON file and subsequently loaded into the SQL database. This count was confirmed directly by running the scraper end to end rather than assumed.

#### **4. Database Schema Design**

Product data is stored in a SQLite database rather than a hosted server such as MySQL. SQLite was chosen because it requires no separate server process, no installation, and no network configuration, and stores the entire database as a single file. This is well suited to a self-contained chatbot project that a grader needs to be able to run locally without setting up external infrastructure.

The products table stores one row per laptop with columns for name, brand, price, processor, RAM, storage, graphics card, rating, and the product URL. Numeric fields such as price and RAM are stored as numbers rather than free text so that the SQL retriever can apply range filters and sorting directly, for example ordering by price or filtering RAM by an exact value, without needing to parse text at query time. Brand values are stored in a consistent uppercase form so that filtering by brand does not depend on inconsistent casing in the scraped source text.

The schema is created once through a schema definition script and populated by a build script that reads the scraped JSON file and inserts each product as a row. This build step only needs to be rerun when the underlying scraped data changes, and the resulting ecommerce.db file is queried directly and repeatedly at runtime by the SQL retriever.

#### **5. FAQ Dataset Preparation**

The FAQ dataset consists of question and answer pairs covering returns, warranty claims, delivery timing, open box and self pickup delivery, order tracking, cash on delivery, installment plans, damaged item handling, and customer support contact, all drawn from priceoye.pk's own policy pages. Each question is embedded once, offline, using a sentence embedding model, and the resulting vectors are saved to disk so they do not need to be recomputed on every application run. At query time, only the user's question needs to be embedded and compared against this precomputed set, which keeps the FAQ lookup fast.

#### **6. Semantic Routing Approach**

Routing between the SQL path and the FAQ path is handled by a semantic router rather than an agent or a tool-calling model. The router defines a small set of example utterances for each route,

ten example product queries for the SQL route and ten example policy queries for the FAQ route. Each set of examples is embedded using the all-MiniLM-L6-v2 sentence embedding model, and the mean of each set's embeddings is treated as that route's centroid in vector space.

When a user sends a message, the message itself, prefixed with recent conversation context where available, is embedded using the same model. The cosine similarity between the query embedding and each route's centroid is computed, and the route with the higher similarity is chosen. If the difference between the top two route scores is smaller than a fixed margin of 0.05, the router treats the query as needing both sources and returns a BOTH decision instead of a single route. Because this entire decision is a deterministic vector comparison with no model generation involved, the routing behaviour is fully inspectable and reproducible, which is why it is shown directly in the debug panel rather than hidden inside a black box.

## **7. Retrieval Logic**

### **SQL Retrieval**

Rather than building a general natural language to SQL model, which would be difficult to make reliable and harder to justify, SQL retrieval is implemented as a structured slot extraction step. The user's query is scanned for known brand names, price bounds expressed as under, above, or between, RAM values expressed in gigabytes, processor keywords such as i3, i5, i7, or Ryzen, sorting intent such as cheapest or most expensive, and mentions of graphics or gaming that imply a dedicated GPU is required. Each recognised slot is converted into a parameterised SQL condition, and the resulting WHERE clause, sort order, and row limit are assembled into a single SELECT statement that is executed against the products table using parameter binding rather than string concatenation, which avoids SQL injection risk from user input.

This approach deliberately favours explainability over generality. Because every filter is extracted through an explicit, named rule, the exact SQL statement and the filters that produced it can be shown in the debug panel for every query, which makes the system's behaviour auditable in a way that a full NL2SQL model would not be.

### **FAQ Retrieval**

FAQ retrieval embeds the user's query, again combined with recent context, and computes cosine similarity against the precomputed FAQ question embeddings. The two highest scoring FAQ entries are considered, but only entries whose similarity score is at least 0.35 are returned as matches, which prevents the system from returning a policy answer that is only weakly related to what the user actually asked. If no FAQ entry clears this threshold, no FAQ content is included in the response.

## **8. Conversation Memory Design**

Conversation memory is implemented as a simple sliding window holding the three most recent user and assistant turns, using a fixed size deque so the oldest turn is automatically dropped once a fourth turn arrives. Only the user's previous messages, not the assistant's previous answers, are joined together to form the context string used for routing and retrieval. This was a deliberate choice: including the assistant's own generated answers in the embedding context risks the context drifting toward whatever phrasing the assistant previously used rather than staying anchored to what the user actually asked, which could bias later routing decisions. The three turn window was chosen as a practical balance between letting the user ask short follow up questions such as a cheaper one or does it have SSD storage, and keeping the embedded context short enough that it does not dilute the similarity signal used for routing.

## **9. Handling of Combined Queries**

Queries that plausibly require both product information and policy information, for example asking for a laptop recommendation and the store's return policy in the same message, are handled through the BOTH decision produced by the semantic router. When the router returns BOTH, the SQL retriever and the FAQ retriever are both executed independently against the same query, and their results are formatted separately before being combined into a single response. No additional coordination logic or language model call is required to decide this split, since the decision was already made at the routing stage based on the closeness of the two route scores.

## **10. Answer Generation and UI Decisions**

Once retrieval results are available, a final answer is produced in one of two ways. If a Groq API key is configured, the retrieved product rows and FAQ answers are passed to a large language model purely as a phrasing step, with an explicit instruction not to introduce information beyond what was retrieved. This call has no influence on which route was chosen or which rows or FAQ entries were retrieved, so it sits outside the routing and retrieval boundary rather than acting as an agent making decisions. If no Groq key is configured, or the call fails for any reason, the system falls back to a plain templated answer built directly from the retrieved rows and FAQ text, so the chatbot remains fully functional without any external language model dependency.

The interface itself is built with Streamlit's chat components, showing the conversation as a sequence of user and assistant chat bubbles. A sidebar allows the debug view to be toggled on or off and provides a button to clear the conversation and reset memory. When the debug view is enabled, each assistant message is followed by an expandable panel showing the exact routing decision, the similarity scores for each route, the extracted SQL filters and the SQL statement that

was executed, and the FAQ matches with their similarity scores, so the reasoning behind every answer can be inspected rather than taken on faith.

As a concrete example, the query cheapest laptops was routed to SQL with a similarity score of 0.613 against the SQL route and 0.0557 against the FAQ route, since no prior conversation context was present. The extracted filter was a price ascending sort with no brand, RAM, or processor constraint, producing the statement `SELECT name, brand, price, cpu, ram_gb, storage, gpu, url FROM products ORDER BY price ASC LIMIT 5`. This returned five real laptops from the database ordered from lowest to highest price, led by a Lenovo V15 at Rs 125,499, with no FAQ matches returned since the query carried no policy intent.

### Key runtime parameters

The following parameters govern the behaviour of the routing and retrieval components and were kept explicit and easy to locate in the codebase rather than buried inside larger functions, so they can be tuned without touching the underlying logic.

| Parameter                         | Location                 | Value            |
|-----------------------------------|--------------------------|------------------|
| Conversation memory window        | conversation_memory.py   | 3 turns          |
| FAQ similarity threshold          | faq_retriever.py         | 0.35             |
| FAQ results returned              | faq_retriever.py         | Top 2            |
| Route closeness margin (for BOTH) | semantic_router.py       | 0.05             |
| Embedding model                   | router and faq retriever | all-MiniLM-L6-v2 |
| Products in database              | ecommerce.db             | 204              |

### Debug panel output for the example query

| Field          | Value                                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------------|
| query          | cheapest laptops?                                                                                     |
| context_used   | (empty, first turn)                                                                                   |
| route_decision | SQL                                                                                                   |
| route_scores   | SQL: 0.613, FAQ: 0.0557                                                                               |
| sql_filters    | {"sort": "price_asc"}                                                                                 |
| sql_query      | SELECT name, brand, price, cpu, ram_gb, storage, gpu, url<br>FROM products ORDER BY price ASC LIMIT 5 |

|             |      |
|-------------|------|
| faq_matches | null |
|-------------|------|

## **11. Deployment**

The Streamlit application is run locally and exposed to the internet using ngrok, which creates a temporary public forwarding URL pointing to the local Streamlit port. This avoids the need to deploy the application to a hosting provider for the purposes of a live, time-boxed demonstration, while still allowing the chatbot to be accessed and tested from outside the local machine during the online session.

## **12. Limitations**

- SQL retrieval relies on keyword and pattern based slot extraction rather than a full natural language understanding model, so queries phrased in unusual or indirect ways may not have their filters extracted correctly.
- The semantic router's routing quality depends on the coverage of its example utterances; queries that fall well outside the style of the SQL and FAQ examples may be routed less reliably.
- The FAQ similarity threshold of 0.35 is a fixed value chosen empirically rather than tuned against a labelled evaluation set, so it may occasionally exclude a relevant but loosely phrased match, or include a marginal one.
- Conversation memory only retains the three most recent turns and only the user's side of each turn, so references to information further back in a longer conversation are not resolved.
- The scraper depends on the current HTML structure of priceoye.pk; if the website changes its listing layout, the extraction logic would need to be updated.
- The ngrok forwarding link is temporary and changes each time the tunnel is restarted, so it is only suitable for a live session rather than a permanent deployment.

## **13. Challenges Faced During Development**

The majority of practical difficulty in this project came not from the chatbot logic itself, which worked as designed once connected to real data, but from environment and dependency conflicts encountered when running the application locally on Windows. Several distinct issues were diagnosed and resolved in sequence.

The first issue was a mismatch between the Python interpreter used to install packages and the interpreter used to run the application, which produced a module not found error for Streamlit despite it having been installed successfully. This was resolved by explicitly invoking pip and the application through the full path of the intended Python interpreter rather than relying on a bare python command, which could resolve to a different installation on the system path.

The second issue was a conflict between an installed TensorFlow build and a newer version of NumPy, surfaced through the transformers library attempting to import a TensorFlow backend that was never actually required by this project, since sentence-transformers only depends on PyTorch. This was resolved by uninstalling TensorFlow and its Keras dependency entirely, which removed the failing import path without affecting any functionality actually used by the chatbot.

The most difficult issue was a silent crash that occurred only when the application was run through Streamlit, with no Python exception printed to the terminal, the process simply exiting after roughly twenty seconds and the browser tab left on a permanent loading state. Running the same imports directly outside Streamlit succeeded without error, which narrowed the problem to something specific to Streamlit's execution environment rather than the machine learning libraries themselves. The Streamlit file watcher was disabled as a first hypothesis, since it is known to conflict with native libraries on Windows, but this did not resolve the crash. Running the application with Python's fault handler enabled finally produced a real, low level stack trace, which showed a fatal access violation occurring while importing pyarrow, a dependency pulled in indirectly through the transformers and datasets libraries rather than used directly by this project. Reinstalling pyarrow resolved the crash immediately, indicating the installed copy had a corrupted or incompatible native extension rather than there being an underlying design fault in the chatbot itself.

These issues, while time consuming, are documented here because diagnosing them required forming and testing a specific hypothesis at each step, from interpreter mismatches, to unnecessary TensorFlow imports, to a genuine native library crash, rather than applying a single fix. Each hypothesis was checked against direct evidence, such as interpreter paths, standalone import tests, and a fault handler stack trace, before moving to the next, which kept the debugging process grounded rather than speculative.

## **14. Conclusion**

The completed system meets the requirements of scraping structured product data from a real Pakistani e-commerce website, storing it in a SQL database, preparing a genuine FAQ dataset from the same website, and routing user queries between SQL retrieval, FAQ retrieval, or both using a semantic router built entirely from plain embedding similarity comparisons, without any agent



framework or tool-calling system. Conversation memory, combined query handling, and a transparent debug view were implemented as core parts of the design rather than added afterward, and the application is deployable through Streamlit and reachable externally through ngrok for live demonstration.